Find the Promos

Ayush Raj · IIT Guwahati · Chemical Engineering · Roll `<NUMBER>` — **Prototype** `<link>` · **Repo** `<link>` · **Raw outputs** `<link>` · **Video** `<link>` · Detail in `docs/assumptions.md`, `docs/ai_mistakes_log.md`, `outputs/eval_report.md`.

Why Problem 1

The brief names the bottleneck: an editor watches every episode and picks promo moments by hand, against a weekly release cadence — a recurring cost with a checkable output. I scored all three problems against the five lenses plus feasibility and measurability (`docs/02_decision.md`); only valuing raw feasibility above everything flipped the winner. The risk I accepted is that it's the obvious pick, so I differentiated on method rather than topic.

Assumptions where the brief was silent

A promo is a **cut list**, not a rendered clip. 20–60s and "hook inside 3 seconds" are **hard constraints in code**. Timestamps are **relative to each episode's start** (verified: all 50 begin at `00:00.00`). Splicing is allowed within an episode, but only **one** promo is built across episodes — stitching across 50 needs character identity this corpus can't support. Episode runtimes contradict the "1–2 minute" framing (median 118.9s, **12 of 50 over 150s**, max 200.8s); I treated the data as truth. There is **no ground truth** for "good promo", so weights are hand-set and documented, never described as learned.

How it works

Code owns time; the model owns judgment. The model never sees a timestamp and never writes one — it reads beats as `[b07] JULIANA (desperate): "..."` and answers in beat IDs, and code looks each ID up in the parsed table. That makes a hallucinated timestamp impossible rather than merely unlikely.

Parse *(code)* — 50 files → **3,727 beats**. Reconciles labels vs timestamps vs beats and aborts on mismatch; reports **22 source hazards**; merges **64 speaker labels → 58 characters**, stripping an honorific only when the bare name is attested elsewhere, so `LORD` and `LADY ASHCROFT` stay two people.

Score *(code — the unfair advantage)* — the scripts already label delivery, `(Cold, Commanding)`, `(Screaming, Desperate)`, on **1,472 of 1,487 dialogue beats** across 617 values, so the tool never guesses where the drama is. A hand-written lexicon (90% hit rate) builds an intensity curve and a promo is a shape on it: **18,141** legal windows on real beat boundaries → **3,376** rejected for no hook in the first 3s → **171** shortlisted. 0.8s, offline, free.

Select *(model, 1 call/episode)* — pick among pre-validated windows; write hook, cliffhanger, one-line why, and a hook type from DramaBox's taxonomy (moral outrage / romantic tension / empathy shock / power reversal) so an editor can spread types instead of posting four of a kind.

Splice *(code)* — two-piece promos within one episode, 23–77s of middle removed, rejected unless the pair has an arc: the speaker changes or the intensity rises. **41 of 91** candidates were two-piece.

Validate *(code)* — 6 hard checks, plus duplicate suppression on **visual** text between episodes 2 apart: 37 dropped, of which 10 are genuine cross-episode repeats including episodes 31 and 32, which both close on a branding iron inches from the heroine's eye.

Gate *(model, 1 call/promo)* — a fresh instance sees **only the clip**, no logline or episode number or surrounding beats, and must say who this is, what's at stake, what question remains, and whether it would keep watching.

The tool never posts: it turns "watch 50 episodes and scrub" into "review 47 ranked, timestamped candidates", and the editor's keep/reject log is the dataset that would let the weights stop being my opinion.

Scale and cost

Measured: one 50-episode show = **127 calls, 124k in + 75k out tokens**; at a show a week (2,600 episodes/yr) ~6,600 calls and ~10.4M tokens. **$0 on free tier**, ~**$2.20/yr** at Gemini Flash-Lite rates. **Tokens are not the scaling cost** — the binding constraint is the free tier's 8,000 tokens/minute, which paces a show to ~20 min of wall clock and vanishes on a paid key. The real cost is editor attention, which is why the gate and duplicate suppression exist at all.

How I know it's good

Validators, over all 92 shipped + rejected + flagship promos: `timestamps_exist` **92/92** — zero invented timestamps — and **92/92** on each of the other five. **Blind cold-viewer gate**: 54 tested, **7 failed**, pass rate **0.870**, none untested; per criterion, unresolved-ending 1.00, character-identifiable 0.91, stakes-clear 0.76, **would-keep-watching 0.65**. **Determinism**: two runs produce the identical decision fingerprint `c7b809ad92b976e7`. **Coverage**: 42 of 50 episodes ship at least one promo. **Gold**

set of 6 promos hand-picked before the tool existed: all 6 overlap a tool pick, mean IoU **code-only 0.354 · `gpt-oss-120b` 0.465 · `qwen3.8-27b` 0.359**. **Stability** across the two model families, 50 episodes each: mean IoU **0.50**, identical picks 17/50, **no overlap at all on 8/50**, hook-type agreement **0.48** on a four-way label.

Does splicing cost comprehension? The brief says a promo is *usually* one continuous stretch; the gate shows why. Continuous promos pass at **0.886** (mean score 3.36), spliced at **0.800** (3.10). I had charged a 0.10 standalone penalty for the cut on instinct — this is what justifies it, and no spliced promo reaches the top of the ranking.

What I will not claim. The gold-set mean favours the good model, but one episode drives most of the gap: on episode 2 the code-only run shipped nothing, scoring zero, while on episode 31 the deterministic scorer picked my window almost exactly (0.697) where the model missed it (0.033). **With n=6 this instrument cannot rank these approaches.** What survives is narrower: a *weaker* model is worth no more than no model, so "add an LLM" is a thing to measure rather than a free win; and the model clearly earns its place on three jobs code cannot do — writing copy an editor can scan, labelling hook type, and running the gate that rejected 7 promos. Expanding the gold set is the first thing I'd do with more time, ahead of any modelling. Hook-type agreement of 0.48 and 8 of 50 episodes with no cross-model overlap say a real part of this ranking is model taste, not signal — keep the editor in the loop and treat disagreement as a confidence flag.

Where it fails

1. It cannot see — shot quality, performance and music are invisible in text; I'm reading sheet music and guessing at the performance. **2.** The evaluation is too small to answer its own main question (n=6). **3.** "Would keep watching" is the weakest criterion at 0.65: comprehensible but not always compelling, and that gap is taste. **4.** 8 of 50 episodes ship nothing, losing their best candidate to the gate or to duplicate suppression. **5.** The duplicate check nearly shipped dead — my first threshold (0.45 on dialogue text) could never fire, because across 1,225 pairs dialogue overlap peaks at 0.177: repetition here is visual, not verbal. **6.** The alias map is partly judgment — `ALEXANDER'S MOTHER` / `DOWAGER DUCHESS` / `MATRIARCH` are plausibly one woman, left unmerged for want of evidence.

AI tools used, and where they got it wrong

Claude for code and drafting; Groq-hosted `openai/gpt-oss-120b` in the pipeline, `qwen/qwen3.8-27b` as comparison. Full log: `docs/ai_mistakes_log.md`.

The one that matters. On episode 8, `qwen3.8-27b` chose a promo spanning `00:25.00 → 00:49.60` and named as its hook a beat starting **8.7 seconds before the promo begins**. The hook text was fluent and that moment genuinely exists in the episode — it just isn't in the clip. No reader could catch that; only checking the beat table can, and two validators did. The same model, on a three-beat test, returned `start_beat: b_03, end_beat: b_01` — a promo that plays backwards, in valid JSON.

The one that nearly fooled me, in my own code. When splicing landed, gold agreement jumped 0.46 → 0.519, looking like a win. My IoU used a promo's outer envelope, crediting a two-piece promo with covering the seconds the splice *removes*. Fixed to measure the real segment union; agreement returned to 0.465. Nothing errored. A metric that improves when you add code deserves more suspicion than one that doesn't.